从多轮回放到真实 Agent 任务：三模型推理服务性能与前缀缓存行为研究

技术报告 · 2026-10-20 · 基于已完成实验的离线整理

修订重点：Coding / Work Agent评测方法体系。另存同源PDF版本。

研究对象：DeepSeek-V4.1-Flash、DeepSeek-V4-Flash-0731、Qwen3.8-27B。下文分别简称 V4.1、V4、Qwen；简称不代表额外的模型版本。本报告不是已发表或经同行评审的论文。

目录

摘要

1. 引言

2. 相关工作

3. 方法

4. 实验

5. 实验结果

6. 讨论与局限

7. 结论

数据与复现声明

参考文献

摘要

单轮推理性能不足以描述多轮 Agent 服务：历史输入重复发送、工具执行和上下文管理都会影响吞吐、延迟与任务完成时间。本文先梳理Coding / Work Agent的任务基准、执行架构与验收方法，再结合 AgentX 固定轨迹回放、四道真实代码任务，以及 CUDA Graph 和动态通知的专项分析，研究三个模型部署的实际表现。AgentX 使用同一批 256 个会话、2048 条候选请求，在 B200×4、TP4、正常前缀缓存条件下测量 C16/32/48/56/64；真实任务采用四个独立 Claude Code 客户端，共保留 12 组正式执行、2349 次模型请求。结果表明：在 P90 TTFT≤10 s 且 P90 TPOT≤50 ms 的约束下，V4.1 的最高已测可行并发为 56，V4 和 Qwen 为 48；真实任务中不与模型 API 重叠的工具区间占总耗时的 82.80% ~ 96.79%，推理服务排序不能直接转化为整题耗时排序；当前 Anthropic 兼容层对动态 system 消息的前置合并破坏了长前缀稳定性，已定位三个正式服务中的 46 处相关缓存深跌。本文同时公开脱敏数值、图表生成脚本及证据指纹，并保留样本跳过、异构软件栈、单轮测量和验收范围等限制。

关键词：Agent workload；多轮推理；前缀缓存；TTFT；TPOT；端到端性能；vLLM

1. 引言

推理服务常以请求吞吐、首 token 延迟和生成速度衡量性能。然而，代码 Agent 的一次用户任务可能产生数百次模型调用，并穿插依赖安装、编译、测试和后台任务等待。前一轮的工具结果还会成为下一轮输入，使“请求”与“任务”之间形成明显的层级差异：请求更快，不一定意味着任务更快；高缓存命中率，也不意味着整条工作流已高效运行。

另一类困难来自缓存指标本身。缓存命中率下降可能由上下文压缩、输入增长、消息重排或缓存驱逐引起，表面相似的波谷不一定对应同一根因。直接以命中率高低推断 HBM 容量、以不同基线的相对百分比推断优化的额外收益，都容易混淆工作量、调度和计算路径。

本研究围绕三个问题展开：

1. 在固定多轮回放与明确延迟约束下，三个部署的吞吐—交互速度边界有何差异？
2. 回放中观察到的服务性能优势，能否直接解释真实代码任务的完成时间？
3. 缓存命中条件下的 TTFT 差异及命中率波谷，哪些已有受控证据，哪些仍不能归因？

本文的主要工作是将已有实验统一为可核对的测量口径，而非提出新的推理算法：以固定轨迹描述服务能力，以真实客户端执行描述工作流开销，再用配置消融、安装源码和请求对照缩小根因范围。三类证据分别回答容量、端到端行为和实现机制问题，不将它们混为同一组严格控制变量实验。

2. 相关工作

2.1. 范围与术语：多轮不是单一排行榜

本节重点研究能修改代码、执行程序、操作应用并交付可验证产物的Coding Agent与Work Agent，而非普通聊天机器人。按方法组织43项代表性工作，以代码/终端工程、企业工作流、科研/ML工程为主线，服务观测与可靠性为横向支撑。检索核对截至2026-10-20，依据原论文、会议页面及作者项目；不宣称穷尽文献，也没有复现所有基准或混排模型分数。文献方法索引记录来源与边界。

一条工作需求可以引发多次模型—工具—观察循环，例如ReAct[30]，不要求用户反复发言。用户补充需求的多轮交互、跨会话记忆和多角色协同则是不同变量；多个互不通信的客户端并行也不等于协作。应分别报告用户轮次、模型调用、工具动作与持续时间。

2.2. 统一分类：目标、任务、驱动、状态、负载与判定



图R1：Coding / Work任务主线及六维比较框架。它是本文的组织方式，不是论文共同采用的官方标准；图R1 ~ R3为方法重绘，不是新的实测结果。

维度	需要说明的选择	未说明时容易产生的误判
评测目标	做对任务、推进过程、服务效率、可靠性或安全	把高吞吐当成能力更强
任务与模态	API、代码、终端、网页、桌面、研究；文本或多模态	用文本token吞吐覆盖视觉与工具开销
执行驱动	固定轨迹、Agent自主闭环、模拟用户、生产轨迹	把长度回放当作真实任务执行
状态范围	历史、数据库、文件、外部副作用、语义记忆、KV	将所有状态都理解为可随意删除的缓存
到达与资源	固定并发、外部到达、轮间等待、共享/隔离、预算	把推理并发等同于在线会话容量
判定与重复	答案、测试、环境终态、过程进展、重复试验、故障分类	把自报完成、单轮成功或基础设施失败当作能力结论

这六维需要分别说明。例如，同一个代码任务既可以用于自主解题的能力测量，也可以把采集的请求轨迹固定后用于引擎性能测量，但后一种实验不再评估新模型是否会自行产生正确解法。

2.3. Coding Agent：题库、执行器与验证器必须拆开

Coding Agent不是一次代码补全，而是围绕仓库与运行环境，反复搜索、阅读、修改、执行和检验的过程。评测至少包含三个可独立变化的对象：题库提供问题与初态，执行器决定模型怎样使用工具，验证器决定产物是否合格。换模型时若同时改变接口、搜索策略或重试预算，不能把结果差异全部归给模型。

工作	方法重点	主要验收/比较对象	实验设计启示
SWE-bench[19]	从真实GitHub问题到仓库修复	修改后的代码能否通过测试	冻结仓库与依赖，不按补丁文本相似度判分
SWE-bench Pro[33]	企业级长时、多文件问题	更复杂问题的解决能力	公开、保留和商业集合分开报告
SWE-bench-Live[34]	可更新任务与专用Docker环境	新近、跨仓库问题上的可执行修复	冻结评测快照，更新性不等于绝无污染
Terminal-Bench 2.0[20]	真实终端任务、独立环境和验证测试	代码、环境或终态产物	不应把所有终端任务简化为一份diff
SWE-agent[32]	面向Agent设计ACI，连接阅读、编辑与执行	同类软件任务上的接口效果	工具接口也是控制变量
Agentless[35]	定位→修复→补丁验证的三阶段流程	简单流程与自主Agent的对照	不能预设更多自主轮数就更好
OpenHands[36]	沙盒内统一代码、命令行与网页能力	可扩展Agent及跨任务评价	执行平台不是单一题库
OpenHands-Versa[47]	编辑执行、搜索、视觉浏览和文件访问	Coding向一般工作任务迁移	通用性需要跨任务验证

SWE-bench类任务重视“问题是否被修复”；Terminal-Bench类任务还包括环境配置、运行程序和产生工件。耗时常含安装、编译、测试和等待，模型只是工作流的一部分。仅统计推理调用会漏掉题目要求真正完成的工作，简单截断工具返回也可能改变解题能力和后续输入分布。

SWE-agent与Agentless提供了重要的方法对照：一个研究交互接口，另一个用固定流程检验是否需要复杂自主策略。模型比较与执行器比较应拆开：先固定执行器换模型，再固定模型比较执行器；分别记录定位错误、修改错误、验证失败、无进展循环和预算耗尽，而不只保留最终成功率。

可复用的Coding协议是：版本化题面/初始源码/依赖与验证器 → 每次试验隔离工作区 → 保留候选、工具和请求轨迹 → 独立副本验收 → 保留失败与停止状态。参考答案和隐藏验收不能进入答题上下文；参考解可用于检查可解性，但不能充当候选。验证器通过也只证明其测试覆盖，不能自动证明全部语义正确。

2.4. Work Agent：从网页动作走向可交付的工作结果

这里将Work Agent定义为通过浏览器、桌面、文件或业务API完成实际工作流程的Agent，而非普通问答。结果可能是更新业务记录、生成表格、完成跨应用协作或提供有依据的信息。评价应沿“业务目标→依赖步骤→环境终态/文件产物”展开，而不是看点击或调用次数是否多。

工作	工作环境与任务结构	主要检查对象	关键区别
WorkArena[37]	ServiceNow企业应用与BrowserGym	知识工作的业务操作	固定业务记录初态与观察/动作接口
WorkArena++[38]	组合检索、规划、推理与操作	多步骤工作流程	难度来自依赖关系，不只是更多token
TheAgentCompany[39]	公司内网站、程序、代码和同事通信	跨应用的实际工作交付	工具成功不等于业务目标达成
CRMArena-Pro[43]	销售、服务与报价等专业场景	业务完成、交互及保密约束	业务交互有状态与规则，不是聊天风格评分
WebArena[16]	可复现网站中的长路径动作	功能性任务终态	浏览器和网页状态属于环境

工作	工作环境与任务结构	主要检查对象	关键区别
VisualWebArena[17]	需要视觉理解的网页任务	视觉落地后的任务完成	视觉和动作成本不能只用文本token解释
OSWorld[18]	真实桌面、文件与跨应用操作	初态对应的执行式验收	截图、GUI和文件恢复纳入协议
AssistantBench[40]	开放网页上的现实信息搜集	答案正确与导航效果	记录内容漂移、检索时间和证据
BrowseComp[42]	持续查找难获取信息	简短可核验的最终答案	浏览检索切面不等于完整办公交付
SpreadsheetBench[41]	Excel需求与变化输入文件	多用例下的表格变换	单个样例正确不足以证明泛化
SpreadsheetBench 2[46]	跨工作表生成、调试和可视化	端到端业务表格产物	不与旧版孤立操作分数直接相减

这条研究线体现了从局部动作到组合流程、从单应用到跨应用、从答案到真实产物的扩展。应同时保留业务对象、文件版本与工具动作，允许不同正确路径，不强制与参考动作序列逐步一致；但权限、保密、不可逆副作用和指定流程约束不能因终态看似正确而忽略。

WorkArena/OSWorld等可控环境适合重置和重复；AssistantBench/BrowseComp依赖开放网页，更容易漂移，需记录时刻、证据及失效原因。模拟业务数据能提高可重复性，但模拟器延迟不是生产API性能。性能测量还应区分推理服务、工具执行、浏览器/桌面观察、跨应用通信与人工/模拟用户等待。

2.5. 科研与ML工程：代码、实验和结论是不同验收层

工作	执行工作	判定方式	避免过度解释
MLE-bench[21]	数据准备、训练和机器学习实验	竞赛任务结果	不遗漏Agent所启动程序的资源
ScienceAgentBench[44]	科研工作流中的可执行任务	Python程序、运行结果及成本	科研步骤完成不等于端到端科学发现
PaperBench[45]	理解论文、开发代码并执行复现实验	分层rubric与经验证的评判器	分项完成度不是整篇复现成功率

与普通代码修复相比，这类Agent会启动长时间运行的程序并管理大量产物。应区分“源码具备实现”“程序运行成功”“数值/实验符合要求”和“结论得到支持”，并记录答题、运行和验收成本。Agent服务与其训练/测试程序若共用资源，还要说明归属和竞争，不能把所有GPU负载都归为推理。

普通聊天和记忆仅作背景：LongMemEval、LoCoMo[22,23]研究跨会话信息保持，不直接验收代码或办公产物，也不是KV缓存命中测试；本节不展开聊天能力排名。

2.6. 执行架构、过程诊断与验收可信度



图R2：SWE-agent的ACI闭环、Agentless的固定流程、OpenHands/Versa的通用沙盒工具架构，依据[32,35,36,47]抽象重绘。独立验收表示评测边界，不声称原项目实现完全一致，也不是本仓库已集成的框架。

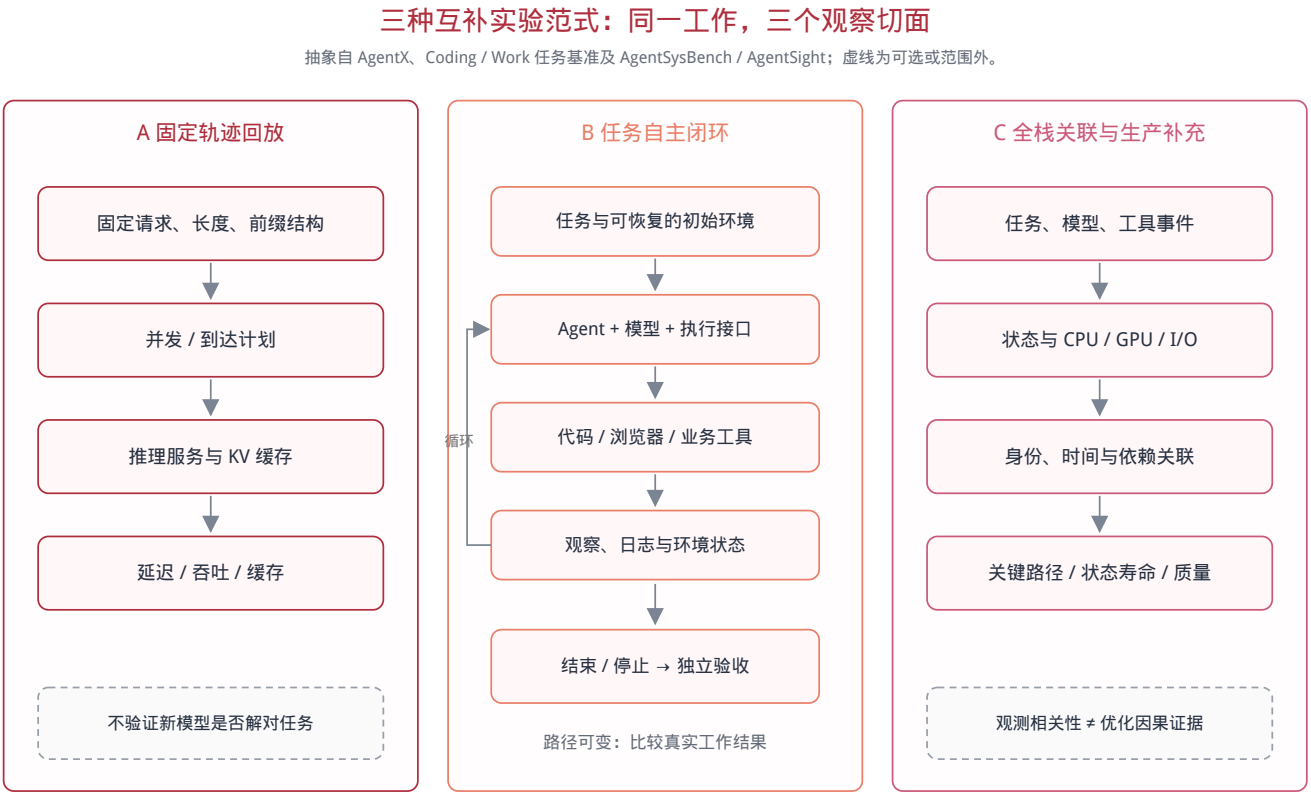
AgentBoard的进展度量[9]可辅助区分“逐步推进”与“反复无效读写”；MultiAgentBench[31]则要求额外检查协作消息、协调与里程碑。对Coding/Work Agent，可以按定位、执行、验证和返工阶段观测，但不能简单累加并行区间作为整题耗时。

τ-bench与τ²-bench[14,15]研究需要用户协作的有状态业务流程，不是简单聊天评分，其模拟用户、规则与共享环境可为Work Agent提供控制方法。ToolBench、StableToolBench、BFCL[11 ~ 13]提供工具组件背景，但函数调用正确不等于工作交付正确。真实API与模拟器之间有稳定性取舍，不能把模拟工具速度称为真实工作流速度。

AgentDojo[24]提醒，读取不可信代码、网页或工具输出时，要同时评估安全和正常任务可用性。一律拒绝工具可能阻断攻击，也阻断工作；这里不把本地通知适配问题直接定性为提示注入漏洞。

2026年的Terminal-Bench难度有效性研究[29]检查“全部Agent失败”背后的题面、验证器和基础设施证据。结论针对特定审阅语料，不代表所有版本存在相同缺陷。共同原则是分开任务失败、环境失败、预算耗尽、验证异常和未验收；发现问题后保留旧轮并重测，不拼接为更好的正常结果。

2.7. 三种实验范式：固定回放、任务闭环与全栈观测



图R3：根据AgentX[3]、Coding/Work任务基准[16 ~ 21,33,34,37 ~ 46]及系统观测工作[1,25]抽象重绘。虚线表示可选或不在该范式中的部分；不是把这些项目组合成已实现的新系统。

范式	固定什么、允许什么变化	能回答	不能单独回答
固定轨迹回放	固定请求集合、前缀结构、输出预算；改变服务条件	引擎吞吐、延迟、缓存与并发敏感性	新模型是否会选对工具、完成任务
Agent自主闭环	固定任务与初态；下一步由当前输出和观察决定	正确性、实际路径、重试与完成时间	在相同请求工作量下谁的引擎更快
全栈观测/生产补充	关联模型、工具、状态与资源事件；保留真实到达或等待	关键路径、状态寿命、控制面及资源关联	仅凭相关性证明某项优化的因果收益

AgentX 适合第一类。本文使用的 Weka loader 保留长度与共享结构，不重建原始代码语义或执行工具。vLLM 的吞吐—交互速度图提供了服务层表达方式[4]，但其成本参数和硬件结果不直接成为本地结果。

还必须将“谁决定下一步”与“请求怎样到达”分开：固定并发回放可以是闭环负载，每个槽位等响应后再发下一条；按外部时间戳或到达率发请求则是另一种负载协议。这里的闭环负载并不等于Agent自主解题闭环。同一批到达时间若被客户端排队拖延，还应报告实际发出时间与计划时间偏差，避免把发生器瓶颈当作服务容量。

2.8. 服务系统方法：复用、等待、依赖与可观测性

工作	方法焦点	应重点测的量	不应混淆的概念
PagedAttention[5]	分页式KV块管理与共享	有效块占用、碎片、共享及吞吐	分配块不等于全部可复用驻留
SGLang[26]	程序化执行及RadixAttention前缀复用	跨调用共享、调度与程序耗时	前缀复用不是语义记忆正确性

工作	方法焦点	应重点测的量	不应混淆的概念
InferCept[28]	外部交互暂停时的状态保留/换出/重算取舍	等待时间、占用、传输与恢复延迟	活跃请求并发不是暂停会话总数
Parrot[27]	通过Semantic Variable暴露请求数据流依赖	应用依赖、完成时间与服务效率	单请求优化不必然优化应用关键路径
AgentSight[25]	边界通信与eBPF事件关联	跨进程调用、系统行为和异常路径	可观测性不是任务验收器
AgentSysBench[1]	工作负载与服务栈联合说明、组件测量及生产补充	端到端时间、状态、资源和控制面	实验室受控观察与生产轨迹不互相替代

AgentSysBench 将研究对象拓展到模型之外的工具、环境与长期状态。其论文和用户提供的 papers.cool 入口是同一项研究[1,2]，不重复计数。本文借鉴其跨层视角，但没有将该论文中的任务感知服务、放置、卸载或工具缓存收益写成本地实测。

这些工作针对的机制不同，不能只汇成一个“缓存优化”：KV块管理、公共前缀共享、暂停状态管理、请求依赖调度和观测关联分别有不同自变量。vLLM的缓存与Graph文档[6,7]提供实现背景，根因判断仍需实际安装源码和控制实验，不能由最新文档反推旧部署行为。

2.9. 统一评测协议与指标选择

基于以上方法，本文建议用“质量约束下的效率”组织结果，保留分层指标而非过早压成一个分数。

层次	核心指标	必须同时注明的条件
任务质量	成功、过程进展、规则合规、失败类别	验收器、任务版本、初始环境、可修改范围
重复可靠性	成功率、pass^k、pass@k	每题重复数、独立重置、采样和预算
请求服务	TTFT、TPOT、吞吐、排队、缓存、错误	实际输出、分位数口径、有效集合、服务配置
整任务效率	完成时间、关键路径、通过任务吞吐	未完成任务、并行重叠、启动/验收是否计入
状态与记忆	记忆正确性、占用、暂停时间、恢复首轮延迟	事实更新、压缩、KV保留/淘汰与权限
资源和成本	CPU/GPU秒、内存、I/O、网络、token及费用	可归属范围、采样、失败开销与辅助调用
安全与稳健性	攻击成功、正常任务可用性、恢复与重试	威胁模型、工具错误、环境漂移和干预范围

τ-bench 的 pass^k 衡量同一任务在 k 次独立试验都成功的一致性；常见 pass@k 衡量至少一次成功。二者方向不同，不能互换。大量请求可以支持请求级分位数，却不能替代独立任务重复；多次都成功也不证明评测器本身正确。

一个可复用的实验闭环是：冻结任务清单与初态 → 校验参考解/验收器 → 固定非研究变量与预算 → 运行并关联模型、工具、状态、资源 → 独立验收与故障分类 → 按任务重复统计 → 在质量和SLA约束下比较效率。缓存清空属于被研究的条件，而不是所有实验必须永远清空：容量曲线需要防止跨点污染，跨会话复用研究则需要明确保留哪些状态。

服务研究宜先固定轨迹隔离引擎因素，再用自主任务闭环检验端到端效果，最后用真实到达与长暂停补充生产适用性。GPU/Node/容器Exporter提供资源证据，只有与任务和调用身份关联后才能用于归因；它们不能生成正确答案或替代验收。

2.10. 横向归纳与本研究的位置

现有工作共同揭示的评测方法趋势可以归纳为三点，而非直接合并各论文的成绩：

1. 从“答案或调用格式正确”走向“可验证终态、过程进展和重复可靠性”，需要把任务、执行器和验证器一起版本化。
2. 从“模型处理请求”走向“模型—工具—状态—环境联合执行”，需要把真实路径、等待和控制面开销纳入计量。
3. 从“某次实验更快”走向“质量与SLA约束内、工作量和资源可比的效率”，需要组合固定回放、任务闭环与跨层观测，而非期待单一基准包办。

本报告的AgentX、四道真实任务和缓存分析只覆盖其中部分服务与观测问题。四题属于代码工程任务，不等于已运行正式SWE-bench或Terminal-Bench；没有执行企业/GUI/科研基准、协作拓扑或安全评测。后文实测只支持实际覆盖范围，文献梳理不代表这些测试已经完成。

3. 方法

3.1. 两层测量与根因证据

服务层固定请求轨迹，改变并发，测量请求延迟与完整回放吞吐；工作流层固定题目与客户端约束，允许模型自主调用工具，测量从客户端启动到交接或停止的实际耗时。两层之间不强行固定调用轮数或输出长度，否则会改变真实 Agent 的决策行为。

根因证据分为三类：

- 描述性观测：曲线、运行均值、区间和资源时序，只描述当前执行。
- 配置消融：保持同一模型和其他主要参数，改变 CUDA Graph 模式，按请求身份严格配对。
- 机制验证：核对实际安装的转换源码，再对捕获的请求做离线转换或仅改变通知的行为对照。

不以资源利用率相关性代替因果证明，不把单次运行内的数千条请求当成数千次独立实验重复，也不使用未完成任务的短路径宣称提速。

3.2. 请求指标

设请求发送、首个有效生成输出、请求结束的时间分别为 t_0、t_1、t_2，实际输出 token 数为 N：

```
TTFT = t₁ − t₀
TPOT = (t₂ − t₁) / (N − 1)，仅统计 N ≥ 2 的请求
P90 interactivity = 1000 / P90(TPOT_ms)
```

TTFT 包含初始排队及客户端可见的传输开销；TPOT 是请求内的平均输出间隔，不是逐 token 间隔分布的 P90。这里沿用参考图的 P90 interactivity 命名，其数值定义为 P90 TPOT 的倒数，不是对每个用户的瞬时生成速度取 P90。回放使用客户端 TPOT；真实任务逐请求 TPOT 来自服务端统计，二者不直接混排比较。

ISL/OSL 分别为每次调用的完整输入、实际输出 token 数。报告展示 P50/P90/P99/Max，分位数采用线性插值，token 数展示为整数；原始数值保留插值结果。输出不超过 1 token 的请求仍参与 TTFT、吞吐和缓存统计，但不参与 TPOT。

服务端 `prefill_to_first_token_s` 表示首次调度到首 token，排除了初始 Queue，但可能包含跨调度迭代的干扰；它不是单个 GPU prefill kernel 的净执行时间。客户端 TTFT−Queue 仍保留传输和计时边界外开销，不与该字段混称为“纯 Prefill”。

3.3. 缓存与吞吐

设输入、缓存命中的输入、输出 token 总量分别为 I、K、O，完整回放耗时为 T，GPU 数为 G：

```
请求命中率 = cached_prompt_tokens / prompt_tokens
全局命中率 = Σ cached_prompt_tokens / Σ prompt_tokens
请求吞吐 = 请求数 / T
输出 token 吞吐 = O / T
总 token 吞吐 = (I + O) / T
Total TPGS = (I + O) / T / G
Output TPGS = O / T / G
```

全局命中率按 token 加权，不是请求命中率的算术平均。Total TPGS 包含已缓存输入，表示逻辑处理吞吐，不是实际新计算的 token 吞吐，更不是 decode 吞吐。缓存输入是完整输入的子集，不能在 I 之外再次相加。

实际服务的 prefix 查询计数可能包含查找或调度重试，不能一概等同于完成请求的命中总量。回放采用完成请求 usage，并用服务端完成请求记录补齐或核对缺失计数；真实任务按已审计的 Anthropic usage 语义还原完整输入，全部请求与后端记录关联。后文不会把重试口径的 counter 直接当作用户请求缓存率。

3.4. 真实任务耗时分解

设 A 为所有模型 API 区间的并集，U 为工具调用区间的并集，E 为该题实际计时窗口。使用如下互斥分解：

```
总耗时 = |A| + |U \ A| + |E \ (A ∪ U)|
工具总耗时 = |U|
API占比 = |A| / |E|
工具总占比 = |U| / |E|
```

工具总耗时可能与 API 重叠，两项占比不要求相加为 100%；堆叠图只画工具不与 API 重叠的部分。API 区间包含请求往返，不等同于 GPU 计算；工具区间包含运行与等待，后台进程也可能在工具返回后继续活动。“未归类”不自动解释为网络通信或空闲。

3.5. 有效性与结果边界

上下文超过当前模型限制的样本跳过，不截断后当作相同工作量；记录跳过数，ISL/OSL 只统计有效样本。并发点之间使用同一模型的固定有效集合；不同模型因 tokenizer、模板和上下文限制不同，有效集合不完全相同。

只有独立验收通过的任务标为通过。自报完成、公开自测、隐藏验收、停止分别记录；验收范围不同的通过结果不能直接合成统一成功率。历史中断轮次与其重测轮次不拼接。全文的“最高并发”均指给定离散测点、数据子集和 SLA 下的最高已测可行值。

4. 实验

4.1. 实验矩阵与服务环境

实验	对象与数据	控制条件	用途
AgentX 并发回放	三模型；2048 条候选；五个并发点	B200×4、TP4、正常前缀缓存、max-num-seqs=256	比较延迟约束内的服务吞吐

实验	对象与数据	控制条件	用途
CUDA Graph 消融	V4.1；公开 112 条、内部 100 条配对请求	C4、正常缓存；主要服务变量仅 Graph 模式	定位执行路径贡献
早期缓存条件复测	V4/V4.1；公开小子集与内部 100 条	C4；零命中各三轮	比较绝对 TTFT 差及波动
真实任务	三模型×四题，2349 次正式请求	每模型四个独立 Claude Code 客户端	端到端耗时与上下文行为
通知机制核验	三个正式服务；另有新版客户端小测试	安装源码、离线转换、真实请求通知对照	区分消息转换与缓存容量问题

这五组实验不是同一个请求集合，不能将早期 C4 的高命中率或均值直接填到后期并发曲线上。本文不重新运行推理；所有图表来自已保存结果的离线分析。

正式真实任务的 DeepSeek 服务安装 vLLM `0.29.1rc1.dev573+ge97573215`，Qwen 派生镜像安装 `0.26.0`；Claude Code 为 `2.1.241`。两个服务栈的版本、模板、权重及执行后端不同，因此三模型对比是部署级比较，不是仅改变模型架构的控制变量实验。版本与适配器指纹见环境证据表。

V4.1 的专项配置包含 `FLASHMLA_MEGA_ATTN_DSV41`、`nvfp4_ds_mla` 和 128-token block；消融保持这些参数不变，只切换 `FULL_AND_PIECEWISE` 与 `FULL_DECODE_ONLY`。Mega 是融合执行路径，不是另一种前缀缓存；本文没有用单项消融证明它解释全部跨模型差距，也不将该后端名称解释为所有 B200 部署唯一可用或其他 GPU 一定不可用。

4.2. AgentX 数据与回放协议

固定选择 256 个会话，每会话前 8 条，共 2048 条候选请求。会话内串行，会话间最多 C 个并行；完成一轮输出后才发送同会话下一轮。不设置缓存命中“预热目标”，从每个会话第一轮开始统计，保留首次未命中。每个并发点使用全新服务，避免复用上一点的 KV Cache。

原始遍历采用较粗粒度，之后补测 C16、C48、C56，形成 C16/32/48/56/64 的最终点集。最终所有点的 `max-num-seqs` 统一为 256，不混入此前较小服务端上限的结果。SLA 同时要求 P90 TTFT≤10 s、P90 TPOT≤50 ms；C64 超标后不继续增加并发。每点只有一次正式测量，不能据此估计运行间置信区间。

模型	上下文上限	有效请求/候选	跳过	有效会话	ISL	OSL
DeepSeek-V4.1-Flash	1048576	2048/2048	0	256	56456/108285/192650/776333	382/1762/7402/32370
DeepSeek-V4-Flash-0731	1048576	2048/2048	0	256	56508/108338/192703/776386	382/1762/7402/32370
Qwen3.8-27B	262144	2033/2048	15	255	56084/104963/169787/202988	381/1760/7485/32370

ISL/OSL 每格依次为 P50/P90/P99/Max，单位为 token。Qwen 的长请求被过滤，跨模型的严格共同有效子集比较尚未补做。两个 DeepSeek 的输出总量各为 1694885 token；Qwen 为 1681345 token。原始输出为 0 的两条候选按接口最低要求设置为 1；各测点有 3 条实际输出≤1 的请求不参与 TPOT。

4.3. CUDA Graph 与缓存条件实验

CUDA Graph C4 消融按会话、轮次和请求来源严格配对。公开集两次各有 115 条 profiling 请求，其中 112 条共有；内部集 100 条全部配对。配对公开请求的缓存 token 总量完全相同，未缓存输入仅差 316 token；内部集输入、缓存及未缓存输入完全一致。以服务端首次调度到首 token 为主要执行口径，同时保留客户端 TTFT 和 Queue。

早期缓存条件实验另保留零命中三轮。公开集对六个“模型×轮次”使用共有 112 个请求键；内部集为 100 条有序请求。正常缓存的历史结果没有同等重复分布。内部集零命中的实际输出总量跨轮次和模型存在差异，不属于完全固定输出的重复试验；其范围只能用于描述已观察波动。

4.4. 真实任务协议与验收

任务	内容	主要环境或操作	可获得的验收范围
1	前缀缓存与 Prefill 调度修复	源码阅读、CPU 测试	隐藏材料未提供；只记录自报与公开测试
2	PP/offload 槽宽与几何处理	源码阅读、边界测试	隐藏材料未提供；只记录自报与公开测试
3	GTCRN ConvTranspose 量化修复	容器、依赖准备、量化及数值验证	独立 C1 ~ C7，含重新量化及 7 个数值用例
4	ViT 量化图冗余操作优化	容器、图优化及产物生成	独立已有产物 C1 ~ C6，不重新量化/推理

每模型启动四个隔离的 Claude Code 副本。客户端使用模型 API、自主调用工具，不另设 token 总预算；每题上限后调整为 200 分钟，包含依赖准备、运行、等待及续跑，服务加载和独立验收另计。V4 任务3/4在新增限时时已运行约 264.35 分钟，保留原时长及中止状态，不能改写成 200 分钟，也不能当作同一限时下的完整失败对照。Qwen 任务4在约 200 分钟停止，退出开销后实际为 200.09 分钟。

V4.1 被外部同卡进程干扰的旧轮已排除，采用全新重测轮；其余仅使用选定正式轮次。Qwen 使用独立兼容模板将客户端 high 映射到权重默认 xhigh；不同模型的推理力度不视为等价。任务3/4实际使用迁移后的 revision3、环境镜像 1.1，不能将旧题面中镜像 1.0 的操作当作实际执行版本。

采集客户端模型请求、工具事件、压缩事件，关联服务端逐请求计时与 token 数据；同时保存 GPU 利用率、分配 KV 块及可归属容器的部分 CPU/I/O 时序。正式请求全部 HTTP 200、全部关联后端且 token 对账一致。HTTP 成功不表示题目通过，工具错误标记也不等于所有测试失败。

5. 实验结果

5.1. AgentX：V4.1 扩展了当前 SLA 内的可行并发

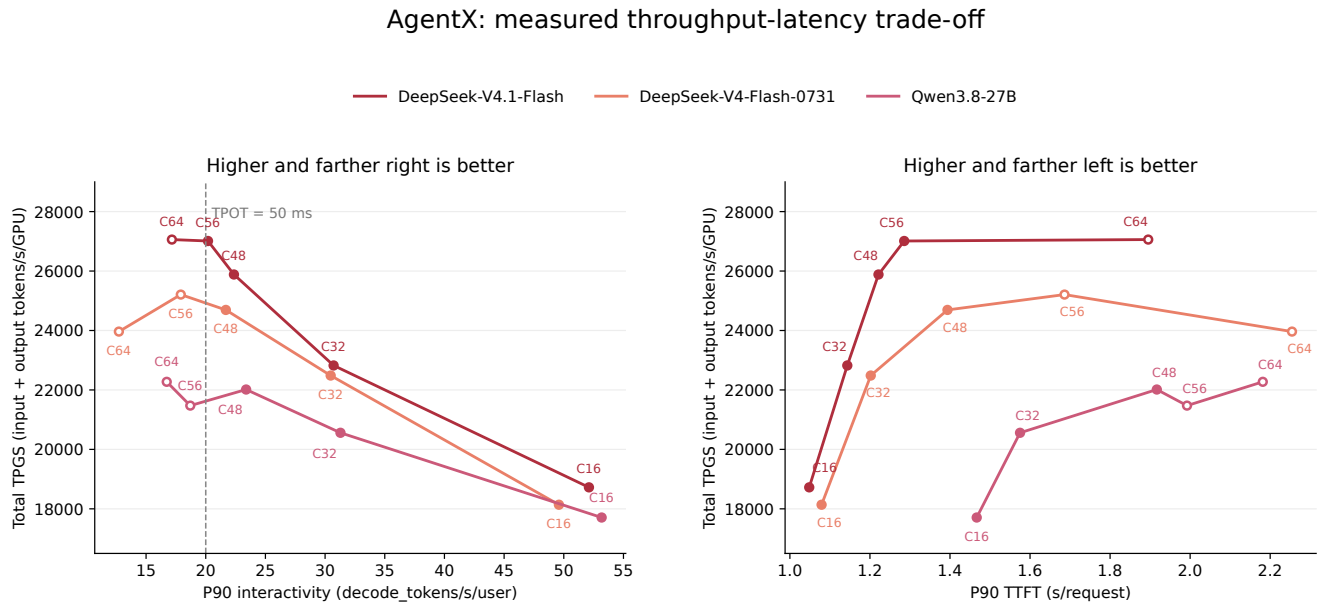


图1：左图越靠右上越好，右图越靠左上越好。实心点同时满足两项 P90 SLA，空心点超标。连线只连接实测点，不表示中间并发已经测量，也不是置信区间。右图仅展示约 1 ~ 2.3 s 的实测 P90 TTFT 范围；10 s 阈值在范围之外，但达标判断仍使用两项阈值。

模型	C	P90 TTFT(s)	P90 TPOT(ms)	Total TPGS	全局 Cache-hit(%)	SLA
V4.1	16	1.048	19.19	18723.12	79.13	通过
V4.1	32	1.143	32.56	22823.58	79.13	通过
V4.1	48	1.221	44.72	25883.64	79.13	通过
V4.1	56	1.286	49.53	27010.95	79.13	通过
V4.1	64	1.895	58.26	27058.94	79.13	超限
V4	16	1.079	20.16	18138.19	77.89	通过
V4	32	1.202	32.82	22484.45	77.85	通过
V4	48	1.394	46.12	24691.91	77.85	通过
V4	56	1.686	55.84	25208.34	77.85	超限
V4	64	2.255	78.61	23963.45	77.85	超限
Qwen	16	1.467	18.81	17708.07	77.92	通过
Qwen	32	1.575	31.96	20558.59	77.83	通过
Qwen	48	1.917	42.77	22014.20	77.63	通过
Qwen	56	1.992	53.47	21473.05	77.31	超限
Qwen	64	2.182	59.77	22273.52	76.97	超限

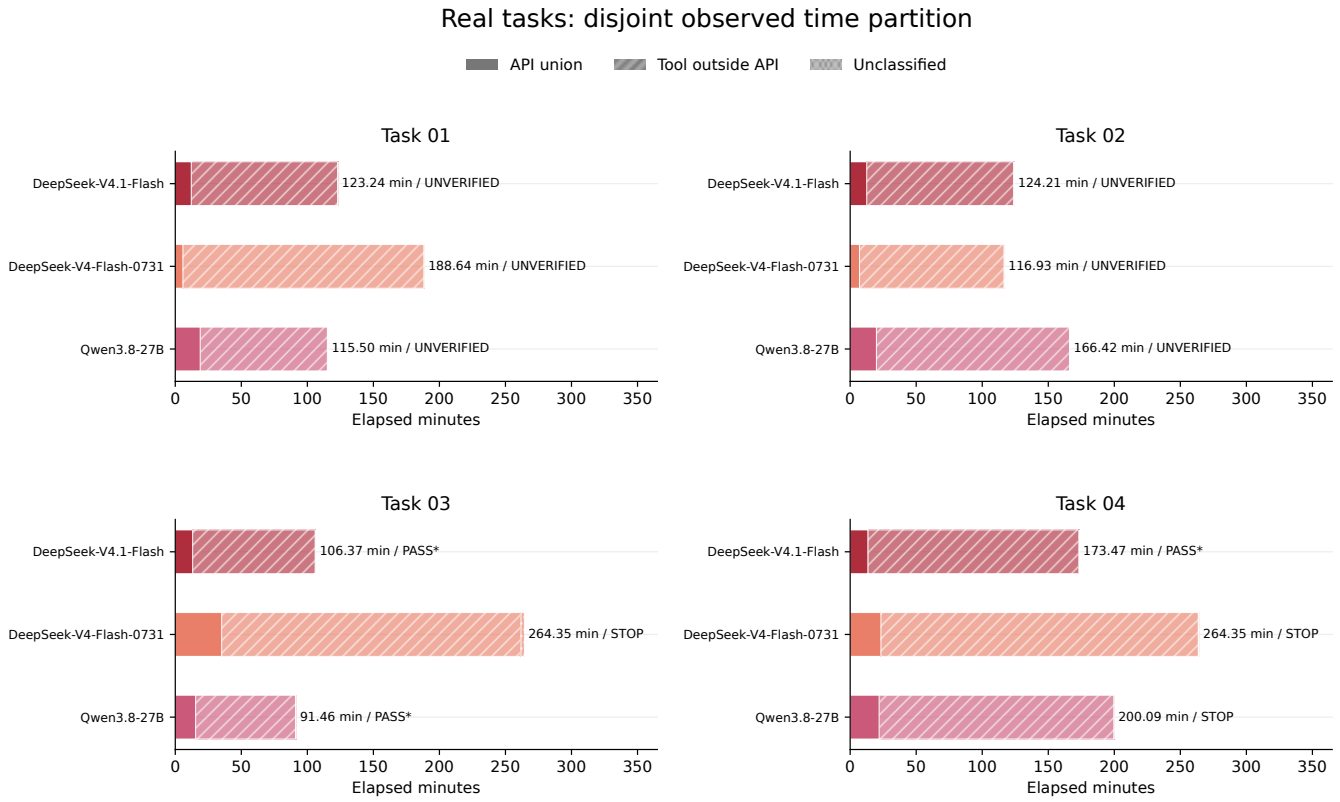
在已测点中，V4.1 最高可行并发为 56，V4、Qwen 为 48。C56 的主要区分项是 TPOT，而非 TTFT。V4.1 从 C56 到 C64 的 Total TPGS 增长不足 0.2%，P90 TPOT 却由 49.53 ms 上升到 58.26 ms，因此更高并发并没有带来有意义的约束内吞吐收益。

这并不表示 V4.1 在所有指标上都最优。例如 Qwen 在 C16/32/48 的 P90 TPOT 均略低于 V4.1，但 TTFT 更高，且有效输入总量不同。不能只按单一指标或未归一化工作量给出统一“速度排名”。

P90 达标也不能代表尾部风险已经消失：V4.1 C56 的 TTFT P99 为 33.054 s、Max 为 45.584 s。若要求改为 P99 TTFT≤10 s，当前 C56 结论不再适用。完整 P50/P90/P99/Max 见分位数表，请求与 token 吞吐见汇总表。

此外，从 C16 到 C64，V4.1 的全局命中率保持约 79.13%，V4 约 77.85%，Qwen 由 77.92% 下降到 76.97%。本组数据没有出现“相对低并发下降至少 5 个百分点”的现象，因此不能声称已找出 HBM 缓存容量的转折并发，更不能把 C56 的 TPOT 边界归因为缓存容量耗尽。

5.2. 真实任务：服务层优势不能直接外推为整题收益



API is not pure GPU time. Tool-total union may overlap API and is listed separately in the tables.
*Task 3: full C1-C7 acceptance; Task 4: saved-artifact C1-C6 only. V4 Task 3/4 are legacy stops beyond 200 min.

图2：按任务分栏比较三个模型。堆叠部分为 API 并集、工具不与 API 重叠的区间和未归类区间；停止或自报状态不等于通过。任务3与任务4的独立验收范围不同。

模型	任务1(min)	任务2(min)	任务3(min)	任务4(min)
V4.1	123.24，自报/隐藏未验	124.21，自报/隐藏未验	106.37，独立通过	173.47，已有产物验收通过
V4	188.64，自报/隐藏未验	116.93，自报/隐藏未验	264.35，历史中止	264.35，历史中止
Qwen	115.50，自报/隐藏未验	166.42，自报/隐藏未验	91.46，独立通过	200.09，限时未完成

十二组执行中，有 3 组通过相应独立验收，6 组自报完成但隐藏未验，3 组停止。任务1/2不能据此判为正确或错误，任务4的通过也不覆盖重新量化和新增推理。V4.1 任务3评测 JSON 和量化、数值子进程记录通过，但未单独捕获评测主进程退出码；这一元数据缺口保留。Qwen 任务3实际评测退出码为 0。验收另计耗时分别为 V4.1 任务3 31.67 分钟、任务4 1.85 分钟、Qwen 任务3 31.58 分钟。

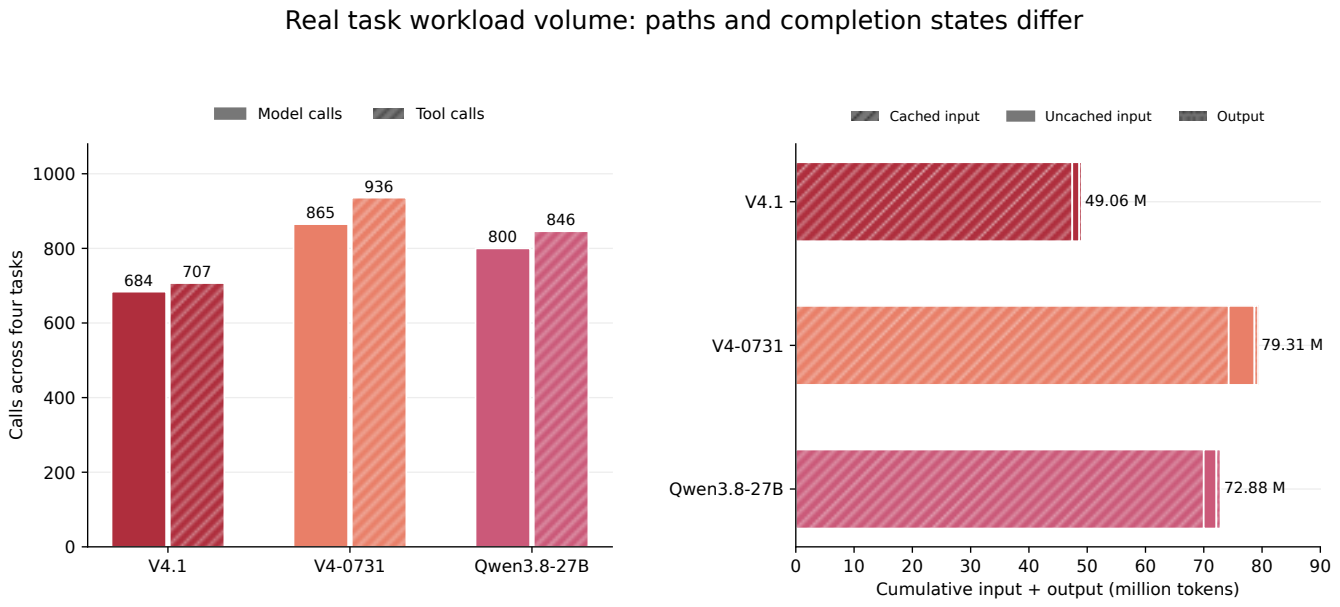
各题 API 区间占实际耗时的 3.07% ~ 16.85%，不与 API 重叠的工具区间占 82.80% ~ 96.79%。这说明当前代码任务的已观测时间主要落在工具和等待区间，但不能进一步未经剖析就认定其中全部是 CPU 计算、环境安装或可消除的空闲。完整 API/工具总耗时及占比见逐题表。

均通过完整验收的任务3给出了一个直接反例：

指标	V4.1	Qwen	Qwen−V4.1
实际总耗时(min)	106.37	91.46	-14.92
API 区间(min)	13.16	15.41	+2.24
工具不与 API 重叠区间(min)	92.82	75.73	-17.09

Qwen 这次整题更快，但 API 区间反而更多；主要差异出现在工具执行路径和等待。因此，不能用本轮整题差异证明 Qwen 推理更快，也不能用 AgentX 中 V4.1 的优势预测它在每道真实题上必定更快。每个模型每题仅一次正式执行，无法估计稳定的任务级 P90/P99 或通用能力排名。

5.3. 调用、Token 与资源：高缓存不等于低工作流开销



Cached tokens are a subset of input. Auxiliary calls are already included. Stopped tasks are retained; this is not a cost or quality ranking.

图3：四道题累计调用量及 token 量。相同模型使用相同颜色，纹理区分类型。缓存输入是输入的子集，已停止任务仍保留其真实消耗，因此不能把本图当作同等成功工作量下的成本排名。

模型	模型调用	工具调用	可识别辅助调用	完整输入 token	缓存输入 token	输出 token
V4.1	684	707	5	48635500	47437184	428107
V4	865	936	8	78695937	74277120	613717
Qwen	800	846	8	72165483	69990816	712522

合计 21 次明确识别的摘要调用已包含于 2349 次模型调用，而非额外相加。客户端另记录 129 个工具 `is_error` 结果；它们不能直接换算成失败测试数。不同模型调用路径、输出量和结束状态不同，累计 token 不是模型质量分数，也不使用客户端对本地模型别名估算的美元字段当作真实费用。

逐题 token 加权缓存率为 90.01% ~ 98.51%，高于 AgentX 全会话首轮计入后的约 77% ~ 79%。两者输入结构、上下文长度及轮次数量不同，这不是缓存实现突然发生同等幅度提升的证据。多轮累计完整输入包含反复发送的历史，不能解释为同等数量的全新信息。

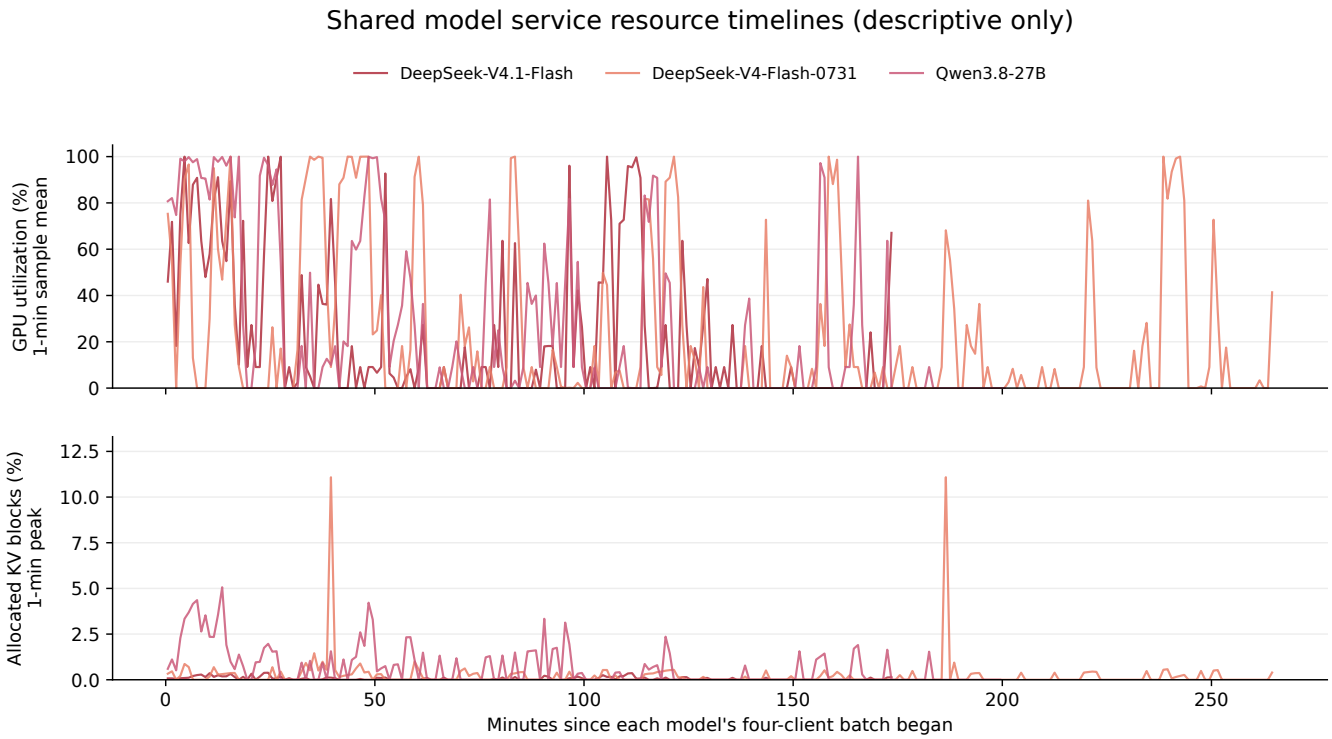
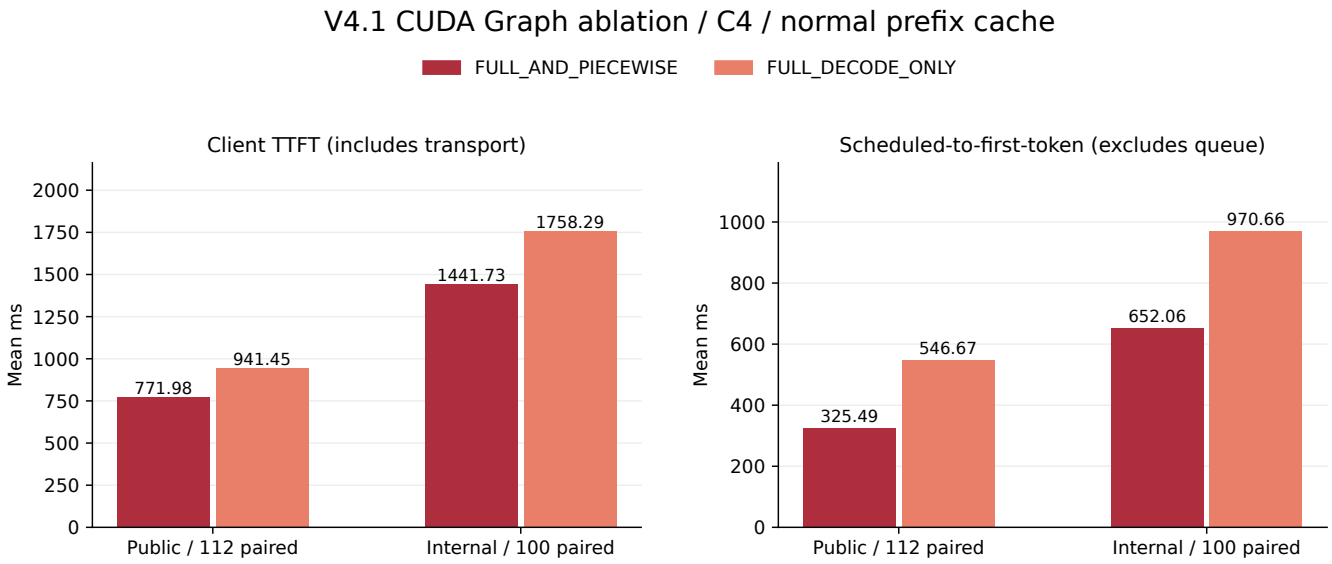


图4：不同模型批次按各自开始时间对齐，并非三模型同时部署。为避免高频采样相互遮挡，GPU 使用每分钟样本均值，KV 使用每分钟峰值，无样本的分钟不插值，原始采样保留在数据中。四题共享同一模型服务，资源曲线不能直接归因到某一道题；GPU 利用率不是 HBM 带宽，已分配 KV 块不是全部可复用缓存驻留。容器 CPU/I/O 仅覆盖可归属子集，CPU 百分比可以超过 100%。

5.4. CUDA Graph：平均收益存在，但不是每条请求固定省时



Means over matched requests; not repeated-run confidence intervals. Server interval excludes initial queue, not inter-iteration interference.

图5：V4.1、C4、正常缓存下的严格配对比较。左图为客户端 TTFT，右图为首次调度到首 token 的服务端时间；二者边界不同。

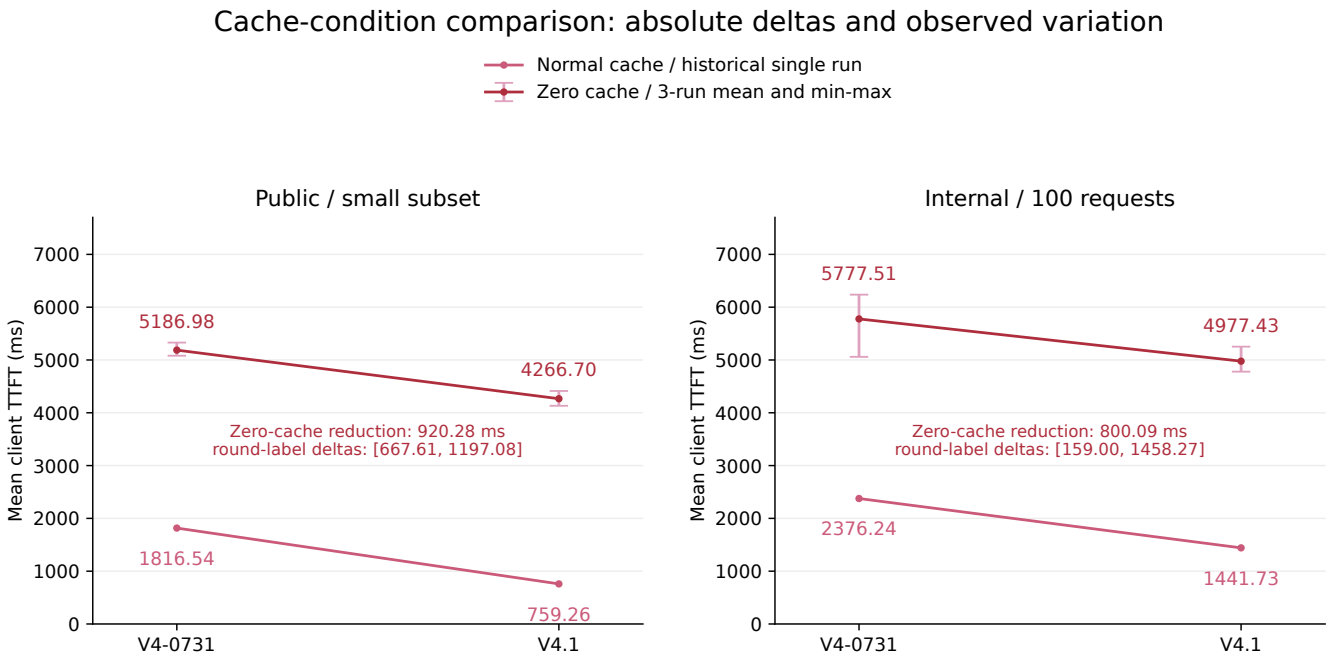
数据集	配对请求	Piecewise 服务端均值 (ms)	Decode-only 服务端均值 (ms)	Decode-only−Piecewise(ms)	配对差值 P50(ms)	客户端 TTFT 差 (ms)
公开 AgentX	112	325.49	546.67	+221.18	-0.02	+169.47
内部 Mongo	100	652.06	970.66	+318.60	-0.25	+316.56

`FULL_DECODE_ONLY` 能正常运行，不存在“强制后必然卡死”的结论。切换为它后，本次配对样本的平均服务端首 token 时间增加，但单请求差值的中位数接近零。均值受部分长尾影响：服务端差值超过 1 s 的请求，公开集为 11/112，内部集为 14/100。因此不能将 221.18 ms 或 318.60 ms 解释为每条请求固定获得的收益。

早期公开集图表的不同请求汇总不能替代本表严格配对后的 169.47 ms；内部集 316.56 ms 是客户端 TTFT 差，不是已排队剔除的 318.60 ms。扣除 Queue 后，客户端差值分别为 169.46 ms 和 352.93 ms，也仍不等同于服务端执行差。

补充的 C1 合成网格中，32K/128K/256K 前缀下客户端“缓存条件额外净收益”约为 29.0/94.1/103.2 ms。它定义为 Piecewise 在正常缓存下的收益减去零命中下的收益，不是正常缓存 TTFT 的全部降幅；尾部实际未命中约 317 ~ 323 token，而不是严格 256 token。该网格不能替代真实 C4 负载，更不能外推为五个并发点均有相同净收益。详情及计算定义见消融补充。

5.5. 缓存条件对比：应看绝对差和重复波动



Historical C4 study, not the 2048-candidate capacity experiment. Min-max is not a confidence interval.
Internal zero-cache output lengths differ across runs; the normal-cache condition has no repeat distribution.

图6：早期 C4 小子集的原始客户端平均 TTFT。正常缓存为历史单轮结果，零命中为三轮运行均值及 Min–Max；误差线不是置信区间，也不是请求级分位数。

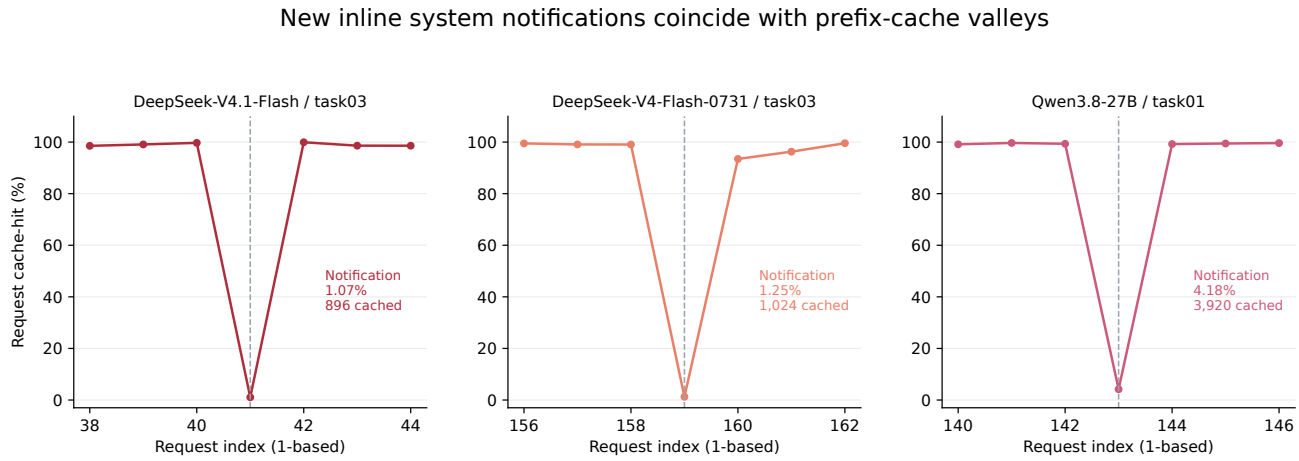
数据集	正常缓存 V4/V4.1(ms)	正常缓存模型差(ms)	零命中 V4/V4.1 三轮均值(ms)	零命中模型差(ms)	同名轮次差的 Min–Max(ms)
公开	1816.54/759.26	1057.28	5186.98/4266.70	920.28	667.61 ~ 1197.08
内部	2376.24/1441.73	934.51	5777.51/4977.43	800.09	159.00 ~ 1458.27

模型差统一为 V4−V4.1，正值表示 V4.1 更低。Min–Max 是按初始、复测1、复测2的轮次标签分别相减得到的范围，不是所有跨轮组合差，也不代表已做同时段严格配对或显著性检验。

正常缓存下的模型差与零命中均值差相近，且位于上述零命中轮次差范围内。现有证据不足以证明“缓存命中给 V4.1 带来了稳定更大的跨模型额外优势”；同样也不能证明两种条件严格等效，因为正常缓存缺少重复、内部输出量不一致。不同分母下相对百分比仍可描述各自比例，但不能单凭 58% 与 17% 就推导缓存的额外因果收益。

零命中的平均排队显著高于正常缓存历史参考值：公开 V4/V4.1 约 848.70/573.68 ms，内部约 755.13/693.22 ms。因此客户端 TTFT 差同时受到排队、执行和请求序列影响，不能全部归给 prefill kernel。专项 Graph 消融的已验证收益，与这里没有证明稳定的“跨模型缓存额外优势”，是两个不同命题，不矛盾。

5.6. 缓存波谷：动态通知前置是已定位的一类根因



Old history, top-level system and tools stay unchanged. Installed adapter source plus offline counterfactual confirms front-merging.
These examples are not compaction boundaries; not every cache valley is a notification.

图7：三处实际通知事件。新增通知时命中 token 本身下降，随后恢复；不是仅因输入变长导致分母增大。

模型/任务	请求序号	通知前命中率	通知加入时	后一请求	命中 token：前→通知
V4.1/任务3	40→41→42	99.70%	1.07%	99.93%	80384→896
V4/任务3	158→159→160	99.06%	1.25%	93.46%	68608→1024
Qwen/任务1	142→143→144	99.34%	4.18%	99.23%	92512→3920

客户端原生事件知道它是 `type: system, subtype: task_notification`，但 API 消息对象只有 `role/content`，通知类别保留在 `<task-notification>` 文本标签中。捕获请求表明，固定 system 和工具定义未变，旧历史忽略 `cache_control` 后保持不变，通知追加在历史末尾。因此不能直接认定是客户端把 system prompt 不断加载到开头。

对实际安装源码的核验发现，兼容层为了满足模板约束，在启用 `merge_inline_system` 时把会话中所有 `role: system` 消息并入开头的 system，并跳过原位置。DeepSeek 未显式提供模板时启用该路径；Qwen 模板禁止中途 system，也触发合并。其效果可简写为：

> 发送时：固定系统指令 → 已有历史 → 新增任务通知
> 转换后：固定系统指令 + 新增通知 → 已有历史

前缀缓存必须从起点连续匹配。动态通知改变较早位置后，后面未改变的长历史也不能继续命中原前缀。使用实际转换函数做离线对照，仅移除新增通知即可恢复前一次 leading system；三个示例的 leading system 字符数分别由 4083/5516/38805 增至 5059/6513/39810。这里是字符数，不是 token，也没有测量修复后的 TTFT 收益。

按“命中率下降至少 30 个百分点、当次低于 50%、排除原生压缩边界”的条件筛选并核对，正式请求中发现 46 处系统消息前置相关深跌：V4.1 8处、V4 26处、Qwen 12处。多数是任务通知，少数为其他系统提醒，不能全部称为 `task_notification`，也不能按次数给模型排序。

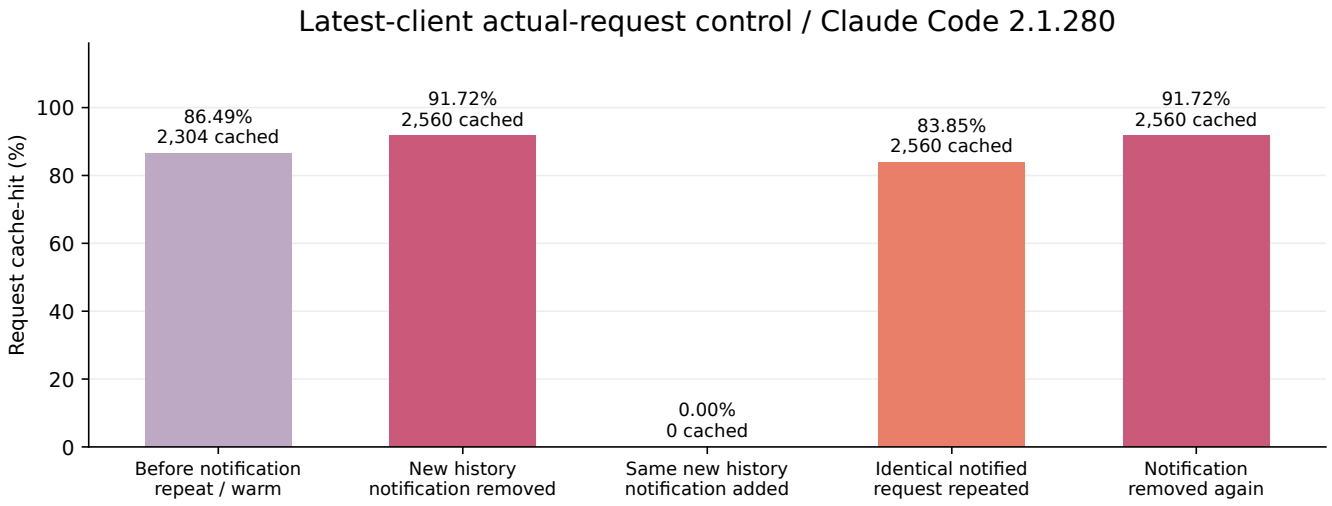
缓存波谷至少应分三类处理：

类别	本次标注数量	证据与解释
上下文压缩	20	原生日志记录压缩，历史被重建；并不都要求达到深跌阈值
system 前缀变化	46	历史追加内嵌 system，经已核验兼容层前置；满足深跌筛选

类别	本次标注数量	证据与解释
输入增长、分母稀释	28	命中率下降≥15个百分点，但命中 token 未减少、输入增加

这 94 处是按指定规则展示的事件，不是所有缓存波动的完整因果普查。没有 preemption 也不能排除空闲前缀被驱逐；仅凭波谷不能证明 HBM 不足。固定 system 在开头本身合理，问题在于动态运行信息与固定指令合并。修复应同时保留通知语义、位置和指令优先级，不能简单删除通知、把所有 system 改成 user，或添加一个服务端未定义的 `subtype` 字段就认为完成。

5.7. 最新客户端验证：现象仍在，部署归因不能越界



Separate gateway deployment; output capped at 16 only for this behavioral check. This is not a throughput benchmark.
Functional cache loss is reproduced; actual A800-route converter source and physical hardware were not independently audited.

图8：另一路用户配置为 A800 的服务入口，以指定来源的 Claude Code `2.1.280` 进行小规模行为验证。该入口经过 LiteLLM `1.98.0`，后端版本接口报告 vLLM `0.28.1rc1.dev688+g5c642796e`；没有重新核验物理 GPU，也没有读取其实际转换源码。

新版实际客户端在临时配置关闭逐轮 token 提醒后，通知前→通知加入→下一请求的命中率为 76.88%→0.00%→79.01%。进一步重放真实请求，仅移除那一条通知时命中 2560 token、91.72%；加入通知时为 0；完全相同的含通知请求再发送后恢复到 2560 token、83.85%；再移除通知仍为 2560 token、91.72%。关键对照保留其余历史与字段，并统一非流式、最多输出 16 token，不执行生成的工具动作。

这证明更新客户端没有消除该链路中的通知相关缓存失效，且不仅是分母效应。它没有独立证明该部署中具体是哪一层前置，也没有评估修复收益；不能把 B200 安装源码的结论和版本直接套到这条链路。默认模式还存在逐轮追加提醒，其缓存变化不单独归因于任务通知。输出 16 仅用于行为对照，不是容量或吞吐测量协议。

5.8. 工具复用：当前认证范围不足以推出普遍收益

认证复用仅覆盖固定只读题面 Read，要求输入、权限、源文件和结果指纹等条件满足。V4 有 18 次题内重复、累计工具时间 1.1253 s，其他模型该认证子集为 0；跨题认证重复均为 0。

这只能说明当前子集未发现大收益，不能推广为“工具缓存没有价值”。Bash、可变工作目录、搜索与网页获取不在认证范围；重复调用也不自动等于可安全复用。与相关工作的搜索工具复用研究相比，本地覆盖不同。即使未来减少累计工具秒数，也需重新检查关键路径，不能直接当作整题减少同等秒数。

6. 讨论与局限

6.1. 已回答与未回答的问题

本研究给出了当前固定子集下的可行并发、真实任务的时间构成，并定位了一个跨三个正式服务出现的消息转换问题。尚不能回答理论最大并发、生产在线会话容量、全数据集上的稳定排名，或修复通知后究竟能节省多少 TTFT/整题时间。

尤其是 C56：其证据是请求级 P90 约束内吞吐较优，不是已证明 GPU 算力、HBM 带宽或缓存容量恰好到达物理极限。并发上升时 TPOT 变差而缓存率稳定，提示应进一步研究执行与调度，但单凭本组曲线不能确定硬件瓶颈。

6.2. 公平性与统计有效性

同一模型的不同并发使用相同有效请求与预算，适合描述该部署的并发敏感性。跨模型还存在模板/tokenizer、上下文过滤、软件版本、执行后端、推理力度及自主执行路径差异。真实题限时规则也曾变化；停止任务是删失/未完成观测，不当作成功耗时或严格同限时失败率。

AgentX 每点一次，真实任务每题每模型一次；零命中仅三轮且输出存在变化。请求级 P99 描述一次运行的尾部，不能代替运行间稳定性；Min–Max 不是置信区间。本报告不使用“统计显著”“严格等效”或通用能力胜出等超出证据的表述。

6.3. 资源归因与共享环境

使用不同 GPU 不保证 CPU、主存带宽、存储或网络互不干扰。GPU 利用率和容器记录可发现时间上的关联，但缺少完整整机背景负载、HBM 带宽及细粒度 kernel 证据；采样、时钟校准和后台进程覆盖也有限。不能把整机指标全部归给自己的任务，也不能因未看到 preemption 就断言不存在任何缓存驱逐。

6.4. 下一步的必要验证

优先进行三项不重复现有指标展示的工作：在共同有效样本上重复关键 SLA 测点并记录运行间波动；对动态通知表示做语义等价的受控修复，验证缓存、TTFT 和端到端收益；结合 CPU/容器、vLLM 与 DCGM/Node Exporter 等采集补齐共享资源和执行关键路径证据。生产到达、长期暂停、状态卸载和搜索工具缓存需要额外工作负载，不假称已由本次连续回放覆盖。

7. 结论

1. 在当前 AgentX 固定候选子集、四卡部署和 P90 SLA 下，V4.1 的最高已测可行并发为 56，V4 与 Qwen 为 48；这不是生产最大容量或 P99 保证。
2. 真实任务主要时间落在工具与等待区间，服务层推理差异不能直接等同于整题收益；完成状态与验收范围必须与耗时一起解释。
3. 缓存性能既取决于执行路径，也取决于消息稳定性。Graph 消融提供了平均首 token 时间收益的证据；动态 system 通知前置提供了缓存失效的机制证据，但修复后的实际加速仍待验证。

数据与复现声明

本仓库提供脱敏数值、8组实验与3组调研PNG/SVG、自动汇总表、文献索引、同源PDF和离线生成/校验脚本。原始提示词、工具输出、题目源码、账号及部署地址不公开，内部数据仅包含去标识的测量数值，不包含样本内容。内容指纹用于由实验所有者核对来源，不使未公开原始材料自动变成可获取的数据。

读者可重新计算已公开测量值的派生指标、配对差及图表，但不能从请求摘要重新计算所有 AgentX 分位数，也不能仅凭公开材料复现真实任务解题结果或独立验证全部原始消息语义。复现说明说明这一区别；证据清单记录输入文件指纹。未启动新 GPU 实验，也未将已有 Web 展示当作新的实验重复。

参考文献

[1] Chaokun Chang, Yukun Zhou, Kaihua Fu, et al. From LLM Inference to Agentic Workloads: Characterization and Implications for Serving Systems. arXiv:2608.15127v1, 2026. PDF · 全文 HTML。

[2] papers.cool. 用户提供的标题检索入口 · 同一论文的单篇入口。文献发现入口，不是独立论文。

[3] SemiAnalysisAI. AgentX Harness，包括 Weka trace 回放及指标说明。

[4] vLLM. Agentic serving: cost efficiency vs. interactivity。本文仅借鉴图形表达，不沿用其成本值或硬件结果。

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180。

[6] vLLM documentation. Automatic Prefix Caching。

[7] vLLM documentation. CUDA Graphs。

[8] Xiao Liu, et al. AgentBench: Evaluating LLMs as Agents. 2023. 原论文。

[9] Chang Ma, et al. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. 2024. 原论文。

[10] Grégoire Mialon, et al. GAIA: a benchmark for General AI Assistants. 2023. 原论文。

[11] Yujia Qin, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. 2023. 原论文。

[12] Zhicheng Guo, et al. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. 2024. 原论文。

[13] Shishir G Patil, et al. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. 2025. 原论文 · 多轮方法说明。

[14] Shunyu Yao, et al. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. 2024. 原论文。

[15] Victor Barres, et al. τ²-Bench: Evaluating Conversational Agents in a Dual-Control Environment. 2025. 原论文。

[16] Shuyan Zhou, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. 2023. 原论文。

[17] Jing Yu Koh, et al. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. 2024. 原论文。

[18] Tianbao Xie, et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. 2024. 原论文。

[19] Carlos E. Jimenez, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. 2023. 原论文。

[20] Mike A. Merrill, et al. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. 2026. 原论文。

[21] Jun Shern Chan, et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. 2024. 原论文。

[22] Di Wu, et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. 2024. 原论文。

[23] Adyasha Maharana, et al. Evaluating Very Long-Term Conversational Memory of LLM Agents. 2024. 原论文。

[24] Edoardo Debenedetti, et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. 2024. 原论文。

[25] Yusheng Zheng, et al. AgentSight: System-Level Observability for AI Agents Using eBPF. 2025. 原论文。

[26] Lianmin Zheng, et al. SGLang: Efficient Execution of Structured Language Model Programs. 2023. 原论文。

[27] Chaofan Lin, et al. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. 2024. 原论文。

[28] Reyna Abhyankar, et al. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. 2024. 原论文。

[29] Edward Lue Chee Lip, et al. What Makes a Terminal-Bench Task Hard? Separating Genuine Hardness from Fake-Hardness on an Adjudicated Agentic Corpus. 2026. 原论文。

[30] Shunyu Yao, et al. ReAct: Synergizing Reasoning and Acting in Language Models. 2022. 原论文。

[31] Kunlun Zhu, et al. MultiAgentBench: Evaluating the Collaboration and Competition of LLM agents. 2025. 原论文。

[32] John Yang, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. 原论文。

[33] Xiang Deng, et al. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?. 2025. 原论文。

[34] Linghao Zhang, et al. SWE-bench Goes Live!. 2025. 原论文。

[35] Chunqiu Steven Xia, et al. Agentless: Demystifying LLM-based Software Engineering Agents. 2024. 原论文。

[36] Xingyao Wang, et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. 2024. 原论文。

[37] Alexandre Drouin, et al. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks?. 2024. 原论文。

[38] Léo Boisvert, et al. WorkArena++: Towards Compositional Planning and Reasoning-based Common Knowledge Work Tasks. 2024. 原论文。

[39] Frank F. Xu, et al. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. 2024. 原论文。

[40] Ori Yoran, et al. AssistantBench: Can Web Agents Solve Realistic and Time-Consuming Tasks?. 2024. 原论文。

[41] Zeyao Ma, et al. SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation. 2024. 原论文。

[42] Jason Wei, et al. BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents. 2025. 原论文。

[43] Kung-Hsiang Huang, et al. CRMArena-Pro: Holistic Assessment of LLM Agents Across Diverse Business Scenarios and Interactions. 2025. 原论文。

[44] Ziru Chen, et al. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. 2024. 原论文。

[45] Giulio Starace, et al. PaperBench: Evaluating AI's Ability to Replicate AI Research. 2025. 原论文。

[46] Jian Zhu, et al. SpreadsheetBench 2: Evaluating Agents on End-to-End Business Spreadsheet Workflows. 2026. 原论文。

[47] Aditya Bharat Soni, et al. Coding Agents with Multimodal Browsing are Generalist Problem Solvers. 2025. 原论文。

以上在线资料核对日期为2026-10-20；文献观点、上游文档和本地实测分开引用。年份按原稿首次发布或所引会议版本标注；[20]明确指2.0，[13]补充V3多轮方法。机器可读书目见原实验书目与调研补充书目。